SECURITY ASSESSMENT

BugPoCer Scan Report

DIFF SCAN · VS MAIN

REPOSITORY	FablesFi/fables
COMMIT	b430a02
BRANCH	unknown
SCANNED	02 November 2026

Prepared by **Olympix** · for **FablesFi**

Table of Contents

1. About

1.1. Methodology

1.2. Disclaimer

1.3. Project Overview

1.3.1. Design Goals

1.3.2. Key Patterns

1.3.3. External Dependencies

1.3.4. Invariants

1.3.5. Security Assumptions

2. Scope

3. Ingested Documentation

4. Summary

5. Findings

5.1. Low

5.1.1. Anchor update samples tick manipulated across transactions

5.1.2. Per-block cached price lets swapper bypass decaying swap-back fee

5.1.3. Sandwich of swap-back trades via cheap fee on price push

5.1.4. Unbounded seeder init price can hurt first LPs

5.1.5. Claim Fee Cut Frontrun By Forced Sync

5.1.6. Sink Change Misroutes Unsynced Revenue

1. About

This report was generated by Olympix's BugPoCer tool. BugPoCer automates end-to-end vulnerability exploitation for smart contracts, moving beyond static warning to deliver concrete, verifiable Proof-of-Concept (PoC) exploits. Built on Olympix's proprietary infrastructure and Intermediate Representation (IR), BugPoCer combines results from multiple analysis engines to identify confirmed security issues. Using an agentic AI framework to synthesize insights across the Olympix toolchain, it enables teams to instantly validate vulnerabilities, measure real-world impact, and confirm that patches are effective.

1.1. Methodology

BugPoCer breaks down a codebase into source *units* (function, contract, or library level). While findings will only be searched for in units within the specified scope, BugPoCer explores all relevant paths within the codebase. It couples this context with an extensive knowledge store to identify potential issues. For each issue, BugPoCer automatically generates a ready-to-run Foundry-format PoC test that reproduces the exploit under realistic attack conditions.

EACH FINDING INCLUDES

- **Vulnerability details**: description, severity, affected lines, and potential impact
- **Exploit demonstration**: a PoC test that reliably triggers the issue

Findings are organised by the defect they represent, not by how confidently BugPoCer could demonstrate it. Several findings that share one root cause and one fix are presented together as a single numbered issue, with each affected site listed beneath it. The issue count therefore reflects the work to be done, and the finding count reflects where it has to be done.

Every finding carries a badge with **BugPoCer's assessment** of it.

Verified findings should be taken into careful consideration. BugPoCer, as any human auditor, can make mistakes, so findings it ruled out may still be worth some review time as a sanity check. They are collected in their own section when included.

BADGES ON EACH FINDING

- **Verified**: BugPoCer wrote a proof-of-concept that runs and demonstrates real harm.
- **Needs Further Review**: BugPoCer could not prove it either way; worth a manual review.

1.2. Disclaimer

This report does not guarantee the discovery of every vulnerability in scope, nor does it ensure that no new issues will arise as the codebase evolves. BugPoCer cannot provide assurances about code added or modified in future iterations. We recommend ongoing security reviews and the use of additional security measures such as independent audits and bug bounty programs.

Findings and suggested remediations in this report are provided for guidance only. Code samples or PoCs are illustrative and may require further testing and adaptation before production use.

This report is intended solely for informational purposes. It should not be interpreted as legal, financial, or investment advice, nor as an endorsement of the audited project.

1.3. Project Overview

Items marked *stated during context validation* were supplied or confirmed by the person who validated this scan's context. Everything else was inferred by BugPoCer.

NAME	Fables
TYPE	AMM / DEX Hooks (Uniswap v4 dynamic-fee hooks with LP ledger, ve(3,3))
DESCRIPTION	Fables is a set of immutable Uniswap v4 hooks for a ve(3,3) DEX on Robinhood Chain. Each hook overrides the LP fee in beforeSwap with either a per-direction fee inside admin bands (keeper pokes, router discounts, continuation surcharge) or Uniswap's StablePair fee around a reference price. It also runs an LP ledger: deposits mint ERC-6909 receipts 1:1 with liquidity, staked receipts route fees to a ve(3,3) distributor, and a claim fee is taken from unstaked fees.

1.3.1. Design Goals

- Hard 10% (100,000 pips) ceiling on the LP fee on every build, and never above the pool cap
- LP principal stays exitable: withdraw cannot be blocked by any role, pause or configuration; unstake never pausable
- Solvent ledger: never owes more than it holds; rounding favours the pool; no fee theft by newcomers
- Fee views match swaps exactly: assembly beforeSwap entries match Solidity twins (feeFor/currentFee/getFee)
- Bounded role powers: keeper pokes, reference moves, minimum fee, discounts and claim fee are rate-limited and stay within admin-set or constant bounds; role setters re-check bounds at execution
- Faithful StablePair math apart from documented changes (10% cap, target floor below 0.9 ratio, bounded reference moves, minimum fee)
- Gas-efficient swaps: assembly beforeSwap entries, packed one-word pool config, Bloom pre-filter for discounts
- Immutable hooks (no proxy, no upgrade) with access control through an OpenZeppelin AccessManager

1.3.2. Key Patterns

- ERC-6909 LP receipts (1:1 with liquidity units, stake-locked)
- OpenZeppelin AccessManaged / AccessManager RBAC (restricted)
- Per-range LP ledger with Q128 fee-per-share accumulators and staking fee routing
- Uniswap V4 Dynamic-Fee Hook (assembly beforeSwap fallback with OVERRIDE_FEE_FLAG)
- Time-bounded pause (hook-wide and per-pool) with unpausable exits

1.3.3. External Dependencies

- Uniswap V4 PoolManager
- Uniswap V4 PoolManager (v4-core)
- OpenZeppelin AccessManager (authority)
- Fables Voter / Ballot Gate (IBallotGate + ve escrow)
- Fables Distributor (IDistributorFables) and FABLES fee sink
- WETH9
- Arbitrum ArbSys precompile (0x64) / Uniswap StablePair fee model

1.3.4. Invariants

High — funds/access risk Medium — functional correctness Low — informational

INFERRED BY BUGPOCER

Properties inferred by automated analysis of code patterns, architecture, and documentation.

78 INVARIANTS

`withdraw` must succeed for a holder of receipts regardless of any role action, pause or configuration. Code: `withdraw` has no `whenNotPaused` / `whenPoolNotPaused` modifier and no role gate; `_withdrawTo` syncs the range (`_syncRange`), settles the user (`settleUser` , with saturating `_accrue` and unchecked `forgone` sums), auto-unstakes any shortfall from staked shares, burns receipts and removes liquidity through `_modify` ; fees are credited, not paid. Admin-settable state it touches includes the claim-fee history walk (`claimFeeFor`), claim-fee recipient, distributor/sinks and the ballot gate; none of it may make `withdraw` revert. `withdrawAndClaim` is pausable and is not the guaranteed exit.

The ledger never owes more than it holds.

Fees earned before a deposit, stake or transfer must not be credited to the newcomer. Mechanism: every share/stake change (`deposit` , `_withdrawTo` , `stake` , `unstake` , `transfer` , `transferFrom`) first calls `_syncRange(id)` (collects v4 fees at current totals via `creditFees` / `_split`) and then `_settleUser` for each affected account, which sets the account's `checkpoint*X128` / `stakedCheckpoint*X128` to the current accumulator levels before the balance changes. A recreated range keeps old accumulator levels; the new depositor's checkpoint is set to them.

Only the hook can initialize its pools; on the Exclusive builds only the ledger can add liquidity.

The LP fee a swap pays (the fee returned with OVERRIDE_FEE_FLAG by every build's beforeSwap entry) must be <= the pool cap and <= 100,000 pips (10%). Fee-engine builds: `FeeRef.keeperFee` clamps to `w.cap()` and `FeeRef.resolve` clamps again after the CF surcharge; `configurePool` requires `cap <= capMax` and `setCategory` requires `capMax <= ABSOLUTE_MAX_FEE` (100_000). Stable builds: `_toPips` caps at `STABLE_MAX_FEE` (100_000) and `configureStablePool` writes `maxFee[id] = STABLE_MAX_FEE` ; the minimum fee (uint16, <= 65,535 pips) and discount must not push the charged fee above the cap.

For the same state and router, the fee a swap pays must equal what `feeFor` quotes; `currentFee` quotes it without a discount. Fee-engine builds: the assembly `fallback` in `FablesFeeFull` must match `FeeRef.resolve` / `_resolveFeeRef` bit for bit (keeper fee, discount for the PoolManager.swap caller, CF surcharge with the CF-word update, cap). Stable builds: the assembly `fallback` around `_stableFee` (and `FablesStableHookLiteTyped.beforeSwap`) must match `getFee`, `currentFee` and `feeFor(key, zeroForOne, router)`, including the target floor, the 10% cap, the minimum fee and the discount.

Pokes move each side's fee within [max(floor, 50% of base), min(pool cap, category cap)] for at most 72 h, raising at most one step per 60 s.

FEE_POKER can turn the continuation fee on or re-anchor it, only at a tick inside the range the call gives and by its deadline, or off.

PAUSER can pause deposits, claims, stakes, transfers and revenue sweeps for up to 7 days per call, clear pokes, switch off the continuation fee and revoke discounts.

ADMIN powers include setting the claim fee, but the quote only names moving LP principal, setting a fee above 10% and un-configuring a pool as listed limits.

LP principal entry: `FablesLedgerV2.deposit(key, tickLower, tickUpper, liquidity, amount0Max, amount1Max, deadline)` registers `_rangeKeys[id]` (id = keccak256(abi.encode(poolId, tickLower, tickUpper))) on first use, calls `_syncRange(id)` and `_settleUser(id, msg.sender)`, increments `_ranges[id].resetSeq` if `totalShares` was 0, raises `_ranges[id].totalShares` by `liquidity`, mints `liquidity` ERC-6909 receipts to `msg.sender`, then `_modify` opens a PoolManager unlock in which `LedgerLibV2._onModify` calls `modifyLiquidity(+liquidity, salt 0)` on the hook-owned position and `_settlePrincipal` pulls `principal = callerDelta - feesAccrued` per currency: ERC20 via `safeTransferFrom(msg.sender, poolManager, owed)` + `settle()`, native via `settle{value: owed}()` from the hook's balance. `_checkBound` reverts if paid principal exceeds `amount0Max` / `amount1Max`. On native pools `_refundNative` sends `address(this).balance - preBalance` back to `msg.sender` last.

Withdrawing and unstaking are never pausable or gated.

Fee collection and split (`LedgerLibV2.creditFees` → `_split`): `_syncRange(id)` (skipped when `_rangeKeys[id].set` is false or `totalShares == 0`, which only rolls `claimFeeSyncIndex[id]`) runs `modifyLiquidity(0)` on the range; `feesAccrued` (fee0, fee1) are minted as PoolManager ERC-6909 claims to the hook. With `total = range.totalShares`: if `total == 0` all fees go to `distributorOwed`; else `stakedCut = _takeStakedCut(fee, total)` goes to `distributorOwed` and raises `accStaked*X128 += cut * 2^128 / totalStaked`; if `unstakedTotal = total - totalStaked == 0` the remainder also goes to `distributorOwed`; otherwise `net = fee - stakedCut`, claim fee `c = net * feeBps / 10_000` (only when `feeBps != 0` and `claimFeeRecipient != 0`) goes to `claimFeeOwed`, and `accFee*X128 += (net - c) * 2^128 / unstakedTotal` (unchecked, wraps). Inputs: fee0/fee1 grow with any swap in the pool (any user), `totalShares` / `totalStaked` move via deposit/withdraw/stake/unstake (any holder), `feeBps` via REVENUE_OPS/ADMIN setters.

settleUser is called before every change to the holder's share count.

The claim fee can be set, at most 20% of LP fees.

The claim fee is charged at no more than the lowest rate in force since a range's last sync, and its recipient is hook-wide.

Rounding favours the pool: fee splits never credit more than was collected, and holder accruals round down.

Holder fee payout: `claimFees(key, tickLower, tickUpper, to, maxFeeBps)` (whenPoolNotPaused) reverts `NothingToClaim` when the caller has zero balance and zero owed, runs `_syncRange` + `_settleUser`, zeroes `position.owed0/owed1`, then `_payout` opens a PAYOUT unlock where `LedgerLibV2._redeem` does `poolManager.burn(hook, currency.toId(), amount)` and `poolManager.take(currency, to, amount)` for each currency. `withdrawAndClaim` does the same after `_withdrawTo`. The hook's PoolManager claim balance (minted in `creditFees`) is the source of every payout.

FablesHook swap fee (`FablesFeeFull` fallback, mirrored by `FeeRef.resolve` / `_resolveFeeRef`): from the pool word `_word[id]`, `fee = base[side]`; if `block.timestamp < expiry` and `poke[side] != 0`, `fee = max(poke[side], max(floor, min(base[side], cap) >> 1))`; `fee = min(fee, cap)`; if `bloom != 0` and the PoolManager.swap `sender` hits the bloom and `_entry[id][sender]` is live for this direction, `fee = min(fee, max(ceil(fee*(10000-min(bps,5000))/10000), floor, minFee))`; if `cfOn`, `fee = min(fee + s, cap)` with `s` from the CF step. Returned as `fee | OVERRIDE_FEE_FLAG`. Inputs: base/floor/cap/category by ADMIN `configurePool`; pokes/expiry by FEE_POKER `pokeFee` / `clearPoke`; entries/bloom by DISCOUNT_ADMIN `setDiscount` / `setDiscountFilter` / `revokeDiscount`; `sender` is whichever contract calls `PoolManager.swap` (any user-chosen router); CF inputs move with the pool tick (any swapper) and time.

Swaps back pay a fee that starts high and falls every block, and the fee is capped at 10%.

Stable builds: the charged fee must be >= StablePair's fee for the same state capped at 10% (i.e. >= min(StablePair fee, 100_000 pips)) — never below it, possibly above it. Once the price ratio to the reference is below 0.9, only the decay target is floored at its value at 0.9 (`_targetFee`); the formula must read and store the real (unclamped) AMM price (as StablePair does) so the movement adjustment in `_decayStartFee` uses real prices; the floor only ever raises the target. (Fix for pre-audit findings 5.1.1 a/b.)

`FablesFeeEngine._word[id]` bases/floor/cap, `FablesFeeEngine._meta[id].category` and `FablesHookCore.maxFee[id]`: set by ADMIN via `FablesFeeEngine.configurePool`, bounded to `key.hooks == this`, dynamic-fee key, existing category, `c.floorMin <= floor <= min(base0, base1)`, `max(base0, base1) <= cap <= c.capMax`, `cap >= 1`; `maxFee[id] = cap`. Callable again on an initialized pool (reconfigure keeps pokes, Bloom filter and CF state). Read by the `FablesFeeFull` fallback (beforeSwap fee), `FablesFeeFull.currentFee`, `FablesFeeFull.feeFor`, `FablesFeeEngine.pokeFee` (floor/cap bounds), discount floor; `maxFee` read by `FablesLedgerV2._checkKey` (deposit, withdraw, withdrawAndClaim, claimFees, stake, setClaimFee, setPoolPaused, pausePool) and `FablesFeeEngine.initializePool`.

`FablesFeeEngine._cf[id]` (mode, kQ8, cfCap, band16, tau/decay1, period, maxAge, anchor, tLast) and the `cfOn` bit of `FablesFeeEngine._word[id]` : parameters set by ADMIN via `FablesFeeEngine.configureCF`, bounded to pool configured, CF off, `mode <= 1`, `1 <= kQ8 <= 256`, `1 <= cfCap <= 2500`, `band16 <= 800`, `5 <= tau <= 300`, `1 <= period <= 60`, `period < min(255, 5*tau)` ; anchor = current tick * 256. Switched on (re-anchored at current tick in [minTick, maxTick], before deadline, kQ8 != 0) or off by FEE_POKER via `FablesFeeEngine.setCFActive` ; killed (off, kQ8 = 0) by PAUSER via `FablesFeeEngine.killCF` . Read by the `FablesFeeFull` fallback when cfOn: anchor A decays toward T = tick*256 by decay1^dt when dt >= period (snap at maxAge) and is written back; x = (zeroForOne ? A - T : T - A), negated in mode 1, minus band16*16; if x > 0, fee = min(fee + min((kQ8 * x * 100) >> 16, cfCap), cap).

Shared state `_ranges[id].totalShares` : raised by `deposit` (+liquidity, after `_syncRange` / `_settleUser`), lowered by `_withdrawTo` (`withdraw` , `withdrawAndClaim` , −liquidity); `resetSeq` is incremented when `totalShares` goes 0→positive (deposit) and when it reaches 0 (withdraw). It is the denominator of the staked cut in `_split` and the `total == 0` branch sends all fees to `distributorOwed` . It always moves together with ERC-6909 `_mint` / `_burn` of the same amount; `transfer` / `transferFrom` move `balanceOf` without changing it.

Shared state `distributorOwed[currency]` and `claimFeeOwed[currency]` (hook-wide per currency, summed across all pools/ranges): raised only inside `LedgerLibV2._split` (from `creditFees` , reached by every `_syncRange` / `_modify` path: deposit, withdraw, withdrawAndClaim, claimFees, stake, unstake, transfer, transferFrom, syncRanges); reset to 0 by `sweepDistributor` / `sweepClaimFee` (anyone, when not paused hook-wide and sink set) and by `_settleSinks` inside `setClaimFee` , `setDistributor` , `setFFFeeSink` . Holder `position.owed0/owed1` is raised by `settleUser` and reset to 0 by `claimFees` and `withdrawAndClaim` . All three are paid from the same pool of hook-held PoolManager claims per currency.

unstake is never pausable, so a holder can always stop routing fees to the distributor.

`FablesLedgerV2._poolClaimFee[poolId]` (bps, history, allTimeMin): set by ADMIN via `FablesLedgerV2.setClaimFee` and by REVENUE_OPS via `FablesLedgerV2.setClaimFeeBps` , bounded by `LedgerLibV2.recordClaimFee` to `bps <= MAX_CLAIM_FEE_BPS` (2000), bps != 0 requires a non-zero recipient, plus `_checkKey` ; each call pushes (fromTs, effective bps) to history and lowers allTimeMin if smaller. setClaimFeeBps reverts while claimFeeRecipient == 0. Read by `LedgerLibV2.creditFees` -> `LedgerLibV2.claimFeeFor` (rate = min over history entries since the range's `claimFeeSyncIndex` , starting from current bps, walking at most 32 entries, else min with allTimeMin; 0 if sinceIndex == 0 or recipient == 0) and `LedgerLibV2._split` (claim cut c = net_unstaked_fee * rate / 10000, rounded down, booked to claimFeeOwed); `FablesLedgerV2._effectiveClaimFee` compared to the user's maxFeeBps in `claimFees` / `withdrawAndClaim` .

`FablesLedgerV2.claimFeeRecipient`: set by ADMIN via `FablesLedgerV2.setClaimFee`, bounded to: cannot return to address(0) once non-zero, `!= address(this)`, `!= poolManager`, `!= distributor`, `!= ffFeeSink`; otherwise any address. Before writing, only the currencies listed in `settleFirst` are swept to the old recipient. Read by `LedgerLibV2.sweepClaimFee` / `FablesNativeETHV2._sweepClaimFee` (destination of all booked `claimFeeOwed[currency]`, native paid as WETH; zero = no-op), `LedgerLibV2._split` and `LedgerLibV2.claimFeeFor` (zero recipient = rate 0), `FablesLedgerV2.setClaimFeeBps`, `FablesLedgerV2.setFFFeeSink` and `LedgerLibV2.validateDistributor` (exclusions).

`FablesLedgerV2.distributor` and its cached `FablesLedgerV2.distributorFF`: set by ADMIN via `FablesLedgerV2.setDistributor` after sweeping `settleFirst` currencies; bounded by `LedgerLibV2.validateDistributor` and `_checkSinkTarget` to zero (clears; distributorFF = 0) or a contract with code, `!= ffFeeSink`, `!= claimFeeRecipient`, `!= address(this)`, `!= poolManager`, whose `ff()` returns non-zero (cached as distributorFF). Read by `LedgerLibV2.sweepDistributor` (destination of `distributorOwed[currency]` unless currency == distributorFF, which goes to ffFeeSink; zero destination = accrual waits), `FablesNativeETHV2._sweepDistributor` (native pot wrapped to WETH and sent to distributor), `LedgerLibV2.recordClaimFee` and `FablesLedgerV2.setFFFeeSink` (exclusions).

`FablesLedgerV2.ffFeeSink`: set by ADMIN via `FablesLedgerV2.setFFFeeSink` after sweeping `settleFirst` currencies; bounded to zero, or a non-zero address `!= distributor`, `!= claimFeeRecipient`, `!= address(this)`, `!= poolManager`. Read by `LedgerLibV2.sweepDistributor` (destination of `distributorOwed[distributorFF]`, i.e. FABLES-denominated staked-share fees; zero = accrual waits), `LedgerLibV2.recordClaimFee` and `LedgerLibV2.validateDistributor` (exclusions). The native override `FablesNativeETHV2._sweepDistributor` does not read it.

`FablesStableFee._words[id].cfg` k / optimalFeeE6 / targetMultiplier / referenceSqrtPriceX96 and `FablesHookCore.maxFee[id]`: set by ADMIN via `FablesStableFee.configureStablePool` (once per pool: reverts if the reference is already non-zero; dynamic-fee key, hooks == this, native only on native builds; `k != 0` with no upper bound, `optimalFeeE6 <= 1e4`, `1 <= targetMultiplier <= 100`, reference within StablePair's bounded v4 range; initial `sqrtPriceX96` passed straight to `poolManager.initialize` with no link to the reference; `maxFee = STABLE_MAX_FEE`) and via `FablesStableFee.updateFeeConfig` (k, optimalFeeE6, targetMultiplier with the same bounds; reference kept; fee state reset). Read by `FablesStableFee._stableFee` -> `_getFee` / `_calculateDecayingFee` / `_targetFee` from the `FablesStableHookLite` fallback and `FablesStableHookLiteTyped.beforeSwap`, and by `getFee`, `currentFee`, `feeFor`, `feeConfig`.

Stable reference price (`referenceSqrtPriceX96` in `FablesStableFee._words[id].cfg`) and `FablesStableHookLite.referenceUpdatedAt[id]`: moved by STABLE_KEEPER via `FablesStableHookLite.updateReferencePrice`, bounded to pool initialized, `block.timestamp >= referenceUpdatedAt + REFERENCE_UPDATE_INTERVAL` (60 s), new reference within the validated range, and `calculatePriceRatioX96(new, previous) >= MIN_REFERENCE_RATIO_X96` (0.999 in Q96); k/optimalFeeE6/targetMultiplier and the whitelist bits kept; fee state reset (decaying fee UNDEFINED, cached price 0). No cumulative bound across calls. Read by `FablesStableFee._getFee` (priceRatio = ratio of AMM sqrt price to reference; picks inside-band fee vs close/far boundary fees and which side pays 0), `_decayStartFee`, `_targetFee` (floor beyond ratio 0.9), and views `getFee` / `currentFee` / `feeFor` / `feeConfig`.

`FablesStableFee._entry[id][account]` and the 32-bit Bloom/seed in `FablesStableFee._words[id].cfg` bits [216,256): set by DISCOUNT_ADMIN via `FablesStableFee.setDiscount` (account != 0, bps <= 5000; for bps != 0: dirs in {1,2,3}, expiry in (now, now + 90 days]; pool initialized; minFee unbounded; bps == 0 deletes) and `FablesStableFee.setDiscountFilter` (any uint8 seed, Bloom rebuilt from listed accounts holding an entry); deleted by PAUSER via `FablesStableFee.revokeDiscount` (no pool check). Read by `FablesStableFee._stableFee` -> `FablesStableFee._discounted` when bloom != 0 and fee != 0: fee = min(fee, max(ceil(fee*(1 - min(bps,50%))), minimumFee, entry minFee)), and by `FablesStableFee.feeFor`.

`AccessManaged.authority` of every hook build (set in the `FablesHookCore` constructor): replaceable only by the current authority via `AccessManaged.setAuthority` (AccessManager ADMIN through `updateAuthority`), with no bound on the new address; role-to-selector bindings, role grants and execution/admin delays live in the AccessManager and are changed by ADMIN. Read by the `restricted` modifier on every setter of `FablesFeeEngine`, `FablesLedgerV2`, `FablesStableFee` and `FablesStableHookLite`. The hooks have no proxy or upgrade path; `poolManager`, `weth` and `USE_ARB_SYS` are immutables.

ADMIN is documented as unable to move LP principal, set a fee above 10%, or un-configure a pool.

The claim-fee rate charged on unstaked LP fees is at most 20% (`MAX_CLAIM_FEE_BPS` = 2000), enforced in `LedgerLibV2.recordClaimFee` for both `FablesLedgerV2.setClaimFee` (ADMIN) and `FablesLedgerV2.setClaimFeeBps` (REVENUE_OPS); the rate applied in `LedgerLibV2._split` is `claimFeeFor` over recorded history entries.

`FablesLedgerV2.withdraw` and `FablesLedgerV2.unstake` carry no pause modifier and no role gate (withdraw auto-unstakes the shortfall), whereas `withdrawAndClaim`, `claimFees`, `deposit`, `stake` and receipt transfers are gated by `whenPoolNotPaused` / `whenRangeNotPaused`. The ledger NatSpec documents this as a design property.

A FEE_POKER poke is valid for at most `MAX_POKE_TTL` (72 h) and a raising poke may move the resolved keeper fee up by at most the category `raiseStep`, no sooner than `RAISE_INTERVAL` (60 s) after the pool's last raise (`FablesFeeEngine.pokeFee`, `FablesFeeEngine._raises`, `FablesFeeEngine._meta.lastRaiseAt`).

Each updateReferencePrice requires ratio >= 0.999 and setMinimumFee takes a uint16 (at most 65,535 pips).

`_ranges[ID].totalShares` equals the sum of `balanceOf` over all accounts; `join` and `leave` change both by the same amount.

`_ranges[ID].totalStaked` equals the sum of `_pos[who].staked` over all accounts, and `_ranges[ID].totalStaked` is at most `_ranges[ID].totalShares`. Each account's staked amount is at most its `balanceOf`. `unstake` does not check the amount against the staked balance, so it relies on checked arithmetic.

The CF anchor updates once per period; if no swap updates it for min(255 s, 5 x tau) it snaps to the price.

Switching the CF on via `FablesFeeEngine.setCFActive(poolId, true, minTick, maxTick, deadline)` must require the current slot0 tick inside [minTick, maxTick] and block.timestamp <= deadline, then re-anchor at that tick (`FeeRef.cfRearm`); `configureCF` must revert `CFActive` while the CF is on, so its anchor (set at the current tick) is replaced on switch-on. (Fix for pre-audit finding 5.1.2 'manipulable spot anchor'.) Observation: switching off has no tick/deadline precondition and `setCFActive(true)` also works while the CF is already on (re-anchoring).

Discounts for listed senders never raise a fee or take it below the floor.

A DISCOUNT_ADMIN discount must never raise any fee and never take it below the floor (fee-engine builds: the pool `floor`; stable builds: the pool's minimum fee). The discounted fee must be <= the undiscounted fee and >= min(undiscounted fee, floor).

Staked cut with carried remainder: `LedgerLibV2._takeStakedCut` computes `carried = stakedResidual + mulDiv(mulmod(fee, totalStaked, total), 2^128, total)` and `cut = mulDiv(fee, totalStaked, total) + (carried >> 128)`, storing `uint128(carried)` back into `range.stakedResidual0/1`. The unstaked side floors twice (`_split` comment: dust under one unit per collection stays as hook-held claims). `totalStaked` and `total` are moved by any holder through `stake`, `unstake`, `deposit`, `withdraw` before the next sync.

The claim fee is charged at no more than the lowest rate in force since a range's last sync.

The staked cut carries its remainder to the next collection, and the claim fee rounds down in LPs' favour.

A claim-fee cut applies to the whole backlog; a raise only to later ones.

Claim fee is bounded at 20% of the unstaked LP-fee residue: `LedgerLibV2.recordClaimFee` reverts when `bps > MAX_CLAIM_FEE_BPS (2000)`, and `_split` applies `feeBps` only to `net = fee - stakedCut`.

Pokes move each side's fee within [max(floor, 50% of base), min(pool cap, category cap)] for at most 72 h, raising at most one step per 60 s.

Pool fee configuration bounds: `setCategory` requires `capMax <= ABSOLUTE_MAX_FEE (100_000 = 10%)`, `floorMin <= capMax`, `raiseStep != 0`; `configurePool` requires dynamic-fee key with `hooks == this`, an existing category, `1 <= cap <= capMax`, `max(base0, base1) <= cap`, `floorMin <= floor <= min(base0, base1)`, writes bases/floor/cap into `_word[id]` and `maxFee[id] = cap` (kept pokes, CF and filter). Existing pools keep stored bounds when a category is later changed. ADMIN may move bases anywhere inside [floor, cap] with no raiseStep limit.

Stable-build decay target below the 0.9 window: the formula reads and stores the real price, as StablePair; once the ratio is below 0.9 the target is max(far - close*targetMultiplier/100, edgeFar - edgeClose*targetMultiplier/100) with edge values computed at MIN_FEE_PRICE_RATIO_X96 (0.9 in Q96). The floor is skipped (never reverts) when edgeClose <= 0 or edgeFar <= edgeCloseShare. Inputs: optimalFeeE6, targetMultiplier set by ADMIN; price by any swapper; reference by ADMIN/STABLE_KEEPER.

STABLE_KEEPER bounds: `updateReferencePrice` may move a pool's reference by about 0.1% at most per minute — requires block.timestamp >= referenceUpdatedAt + REFERENCE_UPDATE_INTERVAL (60 s), `_validateReferenceSqrtPriceX96`, and calculatePriceRatioX96(new, previous) >= MIN_REFERENCE_RATIO_X96 (= Q96 * 999_000 / 1e6, symmetric in direction); it resets the fee state and sets referenceUpdatedAt = now. `setMinimumFee` takes a uint16 (at most 65,535 pips), has no rate limit, and resets the fee state only when lowering. STABLE_KEEPER must not move funds or change any other setting.

Stable builds: every swap must pay at least the pool's minimum fee (`setMinimumFee`, default 0 = no minimum) in both directions, inside and outside the band (`_getFee` raises lpFeeE12 to minimumFee*1e6); no discount may go below it (`_discounted` passes minimumFee as the floor of `DiscountLib.applyEntry`); lowering it must restart the falling fee (`_resetFeeState`) and emit `MinimumFeeSet` . `_resetFeeState` and the per-block state write must preserve the minimum-fee bits [240,256).

`FablesFeeEngine._categories[id]` (floorMin, capMax, raiseStep): set by ADMIN via `FablesFeeEngine.setCategory`, bounded to `capMax <= ABSOLUTE_MAX_FEE` (100_000 pips), `floorMin <= capMax`, `raiseStep != 0`; overwrites an existing category without touching pools already in it. Read by `FablesFeeEngine.configurePool` (floor >= floorMin, cap <= capMax at configure time) and `FablesFeeEngine.pokeFee` (poke ceiling = min(pool cap, current capMax); max rise per raising poke = current raiseStep). Not read by the swap entry (`FablesFeeFull` fallback), which clamps only to the pool's stored cap, so lowering capMax leaves existing pool caps and live pokes in force.

Stable pool admin inputs: `configureStablePool` (once per pool, `_referenceOf(poolId) == 0`) validates config, resets fee state, writes cfg, sets `maxFee[poolId] = STABLE_MAX_FEE` and calls `poolManager.initialize` ; `updateFeeConfig` rewrites k/optimalFeeE6/targetMultiplier and resets state; `updateReferencePrice` requires `block.timestamp >= referenceUpdatedAt + REFERENCE_UPDATE_INTERVAL` and `calculatePriceRatioX96(new, previous) >= MIN_REFERENCE_RATIO_X96` , then resets fee state and writes the reference; `setMinimumFee` writes state bits [240,256) and resets fee state only when lowering. `_resetFeeState` sets the decaying fee UNDEFINED, current clock and cached price 0, so the next swap reads a fresh price and restarts the decay from the far-boundary fee.

SEEDER may only initialize a pool the admin has already configured (FablesHook builds), at the price it passes: `FablesFeeEngine.initializePool(key, sqrtPriceX96)` requires `maxFee[id] != 0` (set only by `configurePool`) and native currency0 only on native builds; the hook places no bound on `sqrtPriceX96` . SEEDER must not be able to configure pools or change existing ones (v4 reverts re-initialization of an initialized pool).

> Every role setter writes the values its caller signs (a fee, a duration, a reference, a minimum) and re-checks all of its bounds when it executes (e.g. `pokeFee` ttl/expiry from block.timestamp and raise interval, `setPaused` duration, `updateReferencePrice` step vs current reference and interval, `setMinimumFee` value, `setDiscount` expiry vs now), so a late-executed signed call changes only when those values apply, never what they are or how far they can go. The one price-dependent role call, `setCFActive(true)`, takes tick bounds [minTick, maxTick] and a deadline. Role setters must not derive values from manipulable live state beyond these bounds.

> A transfer of 0 reverts; staked receipts must be unstaked before they move.

> Sink changes: `setClaimFee`, `setDistributor` and `setFFFeeSink` first call `_settleSinks(settleFirst)`, paying the current sinks the booked pots (`distributorOwed` / `claimFeeOwed`) of the listed currencies; everything else, including fees not yet synced into the ledger, is paid to the sinks set when it is swept. Sweeps (`sweepDistributor`, `sweepClaimFee`) are permissionless, pay `distributorOwed[currency]` to `ffFeeSink` when currency == `distributorFF` else to `distributor`, and `claimFeeOwed` to `claimFeeRecipient`; a sweep to a zero target is a no-op keeping the accrual. Native revenue is paid in WETH. The claim-fee recipient is hook-wide (any pool's `setClaimFee` rewrites it for all pools) and never returns to zero once set.

> Shared state claim-fee policy: `claimFeeRecipient` is hook-wide while `_poolClaimFee[poolId]` (bps, history, allTimeMin) is per pool. `setClaimFee` (sweeps `settleFirst`, then pushes a history entry `{fromTs, effective bps}` for one pool and overwrites the hook-wide `claimFeeRecipient`; reverts when trying to set recipient back to 0 once set) and `setClaimFeeBps` (requires a recipient, pushes an entry for one pool, no sweep) are the only writers; `recordClaimFee` rejects recipient == hook, == `distributor`, == `ffFeeSink`. `claimFeeSyncIndex[id]` is written by `creditFees` (every non-idle sync) and by `_syncRange`'s idle branch, always to the pool's current history length.

> Shared state pool word `_word[id]` (FablesFeeEngine): written by `configurePool` (bases, floor, cap), `pokeFee` (pokes + expiry; also `_meta.lastRaiseAt` on a raise), `clearPoke` (clears pokes), `setDiscount` (ORs bloom bits), `setDiscountFilter` (replaces bloom and seed), `setCFActive` (cfOn bit) and `killCF` (cfOn false). CF word `_cf[id]` is written by `configureCF` (full rewrite incl. anchor at current tick; only while off), `setCFActive(true)` (`cfRearm` at current tick), `killCF` (k = 0) and by the `FablesFeeFull` beforeSwap fallback (tLast/anchor update when `dt >= period`, on any swap).

> Shared state pause timers: `pausedUntil` is set to `now + duration` or lifted to 0 by `setPaused(duration <= 7 days)`, and only extended by `pause(duration)` (reverts if `now + duration <= pausedUntil`); `pausedUntilOf[poolId]` likewise by `setPoolPaused` (set/lift) and `pausePool` (extend only). `pausedFor(poolId)` gates `deposit`, `withdrawAndClaim`, `claimFees`, `stake`, and (via range key) `transfer` / `transferFrom`; hook-wide `paused()` alone gates `sweepDistributor` / `sweepClaimFee`. `withdraw`, `unstake` and `syncRanges` read neither.

Native-ETH deposit accounting: on `currency0 == address(0)` pools `deposit` requires `msg.value >= amount0Max`, stores `address(this).balance - msg.value` in the transient `_FOREIGN_NATIVE_SLOT`, the unlock settles `owed` currency0 via `poolManager.settle{value: owed}()` from the hook balance (bounded by `amount0Max` through `_checkBound`), and `_refundNative` sends `max(0, address(this).balance - pre)` to `msg.sender` via a call that reverts on failure. `receive()` accepts ETH only from `poolManager`; native revenue sweeps take ETH to the hook and wrap it to WETH in the same call.

`FablesLedgerV2.pausedUntil` (hook-wide): set by ADMIN via `FablesLedgerV2.setPaused` (duration <= MAX_PAUSE = 7 days; 0 sets it to 0; may shorten) and by PAUSER via `FablesLedgerV2.pause` (1 <= duration <= 7 days, new until = now + duration must exceed the current pausedUntil; no cap on how many times it can be extended). Read by `FablesLedgerV2.paused` / `FablesLedgerV2.pausedFor` -> `whenNotPaused` (`sweepDistributor`, `sweepClaimFee`), `whenPoolNotPaused` (`deposit`, `claimFees`, `withdrawAndClaim`, `stake`) and `whenRangeNotPaused` (`transfer`, `transferFrom`). Not read by `withdraw`, `unstake`, `syncRanges`, or the internal sweeps run by `setClaimFee` / `setDistributor` / `setFFFeeSink` via `_settleSinks`.

`FablesLedgerV2.pausedUntilOf[poolId]` (per-pool): set by ADMIN via `FablesLedgerV2.setPoolPaused` (duration <= 7 days, 0 lifts, may shorten) and by PAUSER via `FablesLedgerV2.pausePool` (1..7 days, only extends past the current value), both requiring `_checkKey(key)`. Read by `FablesLedgerV2.pausedFor` -> `whenPoolNotPaused` (`deposit`, `claimFees`, `withdrawAndClaim`, `stake`) and `whenRangeNotPaused` (`transfer`, `transferFrom` via the range's stored key; an unknown range resolves to the zero key). Not read by sweeps, `withdraw`, `unstake`, `syncRanges`.

`FablesLedgerV2.ballotGate`: set by ADMIN via `FablesLedgerV2.setBallotGate`, bounded to zero, or a contract with code that answers `ballotBacks`, `isGaugeEarning`, `gaugeWeightNow` without reverting and whose `ve()` escrow (non-zero, with code) returns `voter() == gate`. Read by `FablesLedgerV2.stake` (reverts GaugeNotEarning if gate == 0 or `isGaugeEarning(poolId)` is false; compares `gaugeWeightNow` to the caller's minGaugeWeight) and `FablesLedgerV2.syncRanges` (reverts BallotGateUnset if zero; requires `ballotBacks(msg.sender, tokenId, poolId)` per staked range).

Fee crediting in a share change occurs at post-mutation totals, which the quote says is safe only because each caller's earlier _syncRange drained the fees.

Shared state stable fee words `_words[poolId]`: `state` (decaying fee [0,40), cached sqrtPrice [40,200), clock block [200,240), minimum fee [240,256)) is written by the swap path `_stableFee` (only on the first swap of a clock block or after a reset; preserves min-fee bits), by `_resetFeeState` (from `configureStablePool`, `updateFeeConfig`, `updateReferencePrice`, and `setMinimumFee` when lowering) and by `setMinimumFee` (min-fee bits). `cfg` (k, optimalFeeE6, targetMultiplier, reference [0,216); bloom32/seed [216,256)) is written by `_writeFeeConfig` (keeps whitelist bits) and by the discount setters (whitelist bits).

`FablesHookCore.maxFee[id]` is written only by `FablesFeeEngine.configurePool` (= cap, cap >= 1) and `FablesStableFee.configureStablePool` (= STABLE_MAX_FEE), never to 0; `FablesLedgerV2._checkKey` and `FablesFeeEngine.initializePool` treat a non-zero value as "configured". The core NatSpec states the marker is never reset to 0.

Each pause call extends a freeze by at most `MAX_PAUSE` (7 days) from the call's timestamp: `FablesLedgerV2.pause` / `pausePool` (PAUSER) require 1 <= duration <= 7 days and until > current value; `setPaused` / `setPoolPaused` (ADMIN) require duration <= 7 days. The cap is per call; the code sets no limit on the number of successive extensions.

Owed accrual is saturating, with tokens above 2^127 units unsupported, because a revert would block the holder's withdraw.

No hook uses return-delta flags, remove-liquidity or donate callbacks, or hookData.

`LedgerHandler.callCount` is a cumulative counter that only increases, by exactly 1 per handler call, and is never decreased or reset by any handler function.

`ConservationHandler.callCount` and `ConservationHandler.slackGrowths` only increase, by at most 1 per handler call.

`pausesLanded` is a cumulative counter that only increases (by one per successful `pausePool` or `liftPool`).

`fieldViolations` is a cumulative counter that only increases (via ++) and stays 0 if the hook preserves non-poke fields of the pool word.

1.3.5. Security Assumptions

20 ENTITIES

Uniswap V4 PoolManager TRUSTED

- Invokes hook callbacks honestly per IHooks permission flags
- Enforces single-active-unlocker semantics
- Reverts on non-zero net BalanceDelta at unlock exit
- Does not adversarially re-enter hooks

AccessManager authority (AccessManaged.authority) TRUSTED

- decides every `restricted` call (canCall + scheduled delays) on all hook builds
- the current authority can repoint the hook to a different authority contract via AccessManaged.setAuthority, which hands every restricted setter to whoever controls the new authority
- no upgradeability: poolManager, weth and USE_ARB_SYS are immutables; there is no proxy, initializer or storage gap; the only mutable control pointer is the authority

Ballot gate / Voter (IBallotGate at FablesLedgerV2.ballotGate) TRUSTED

- isGaugeEarning(poolId) decides whether stake() is allowed; gaugeWeightNow(poolId) is compared to the caller's minGaugeWeight
- ballotBacks(caller, tokenId, poolId) decides who may call syncRanges for staked ranges
- its answers are read only at entry of stake/syncRanges; zero gate closes both

Revenue sinks: distributor, ffFeeSink, claimFeeRecipient TRUSTED

- distributor receives distributorOwed (staked shares' fees, and all fees of a range with zero shares) via sweepDistributor; native currency paid as WETH
- ffFeeSink receives distributorOwed of the currency equal to distributorFF (the distributor's ff() token); a zero sink makes that accrual wait
- claimFeeRecipient receives claimFeeOwed via sweepClaimFee; native paid as WETH; a zero recipient makes sweeps no-ops and the claim-fee rate 0
- sweeps are callable by anyone and pay to whatever sink is set at sweep time

WETH9 (FablesNativeETHV2.weth) TRUSTED

- native distributor and claim-fee sweeps call weth.deposit() then safeTransfer to the sink
- LP withdrawals and claims on native pools pay native ETH, not WETH

OpenZeppelin AccessManager / Robinhood Chain / protocol-fee controller TRUSTED

- OpenZeppelin (AccessManager, ReentrancyGuardTransient) enforces restricted role bindings correctly
- Robinhood Chain behaves as an EVM chain (ArbSys(0x64).arbBlockNumber() is the stable-fee clock when USE_ARB_SYS)
- The chain's protocol-fee controller may add up to 0.1% per side on top of the LP fee

External sinks: distributor, ballot gate (IBallotGate), ffFeeSink, claimFeeRecipient TRUSTED

- distributor receives staked-share fees via sweepDistributor and answers ff() with the FABLES token
- ffFeeSink receives distributorOwed in the FABLES token
- claimFeeRecipient receives claimFeeOwed via sweepClaimFee (native as WETH)
- ballotGate answers isGaugeEarning/gaugeWeightNow (gates stake) and ballotBacks (gates syncRanges)
- External contracts are out of audit scope

ADMIN (AccessManager ADMIN_ROLE) SEMI-TRUSTED

- Configures pools and fee bands, sets sinks and the claim fee, grants roles, and can rebind role-to-function bindings
- Cannot move LP principal, set a fee above 10%, or un-configure a pool

FEE_POKER SEMI-TRUSTED

- Moves each side's fee within [max(floor, 50% of base), min(pool cap, category cap)] for at most 72 h, raising at most one step per 60 s
- Turns the continuation fee (<= 25 bp, within the pool cap) on or re-anchors it only at a tick inside the call's range and by its deadline, or off
- Cannot leave those bounds

PAUSER SEMI-TRUSTED

- Pauses deposits, claims, stakes, transfers and revenue sweeps for up to 7 days per call
- Clears pokes, switches off the continuation fee, revokes discounts
- Cannot pause withdraw or unstake

DISCOUNT_ADMIN SEMI-TRUSTED

- Lists senders (routers) for a discount that never raises a fee or goes below the floor
- Cannot raise any fee

REVENUE_OPS SEMI-TRUSTED

- Sets the claim fee, at most 20% of LP fees (setClaimFeeBps against the existing recipient)
- Cannot touch principal

SEEDER SEMI-TRUSTED

- Initializes a pool the admin has configured (FablesHook builds), at the price it passes
- Cannot configure pools or change existing ones

STABLE_KEEPER SEMI-TRUSTED

- On stable builds, moves a pool's reference by about 0.1% at most per minute (price ratio >= 0.999 to the current reference)
- Sets a pool's minimum fee (at most 65,535 pips)
- Cannot move funds or change any other setting

Token issuers (pool currencies) SEMI-TRUSTED

- May pause their token; while paused, exits that move a non-zero amount of it wait until it is unpaused
- Tokens must not be fee-on-transfer or rebasing, and supply must not exceed 2^127 units

ADMIN (AccessManager ADMIN_ROLE, role 0) SEMI-TRUSTED
STATED IN VALIDATION

- Findings are: moving LP principal, setting a fee above 10%, un-configuring a pool, and any ADMIN path that blocks withdraw/unstake
- Other in-bounds ADMIN config harm (reconfiguring live-pool bases, ballotGate to 0, repeated setPaused, rebinding selectors/authority) is accepted

Pool tokens / token issuers SEMI-TRUSTED STATED IN VALIDATION

- Issuer may pause its token. While paused, any withdraw, claimFees or withdrawAndClaim that moves a non-zero amount of that token reverts until unpaused; exits that move none of it succeed
- Known behaviour, accepted: a paused token stalls only exits and sweeps that move that token
- If the issuer blocklists only the LP's own address, funds are not stuck because the LP chooses `to` (_checkPayoutTarget refuses only the hook and the PoolManager)
- Revenue sweeps pay one currency per unlock, so a paused token stalls only its own sweep
- Deposits pull principal with safeTransferFrom(msg.sender -> PoolManager) and then settle() inside the MODIFY unlock
- Fee-on-transfer, rebasing and >2^127-supply tokens are unsupported

Users (LPs, receipt holders, swappers, anyone) UNTRUSTED

- deposit liquidity into any range of a configured pool via FablesLedgerV2.deposit (pausable), receiving ERC-6909 receipts 1:1 with liquidity
- withdraw principal via FablesLedgerV2.withdraw (no pause or role gate, auto-unstakes) to any `to` except address(0), the hook and the PoolManager
- claim fees via claimFees / withdrawAndClaim (pausable) with a maxFeeBps bound on the claim-fee rate
- stake / unstake receipts (stake requires an earning gauge; unstake never pausable); transfer unstaked receipts (pausable per range)
- call sweepDistributor / sweepClaimFee for any currency (hook-wide pause only) and force a range sync through these entry points, which prices the backlog at the rate in force then
- swap through the PoolManager on any pool using the hook, paying the fee the hook resolves; choose router/sender and price movement within a block (affects CF anchor and stable cached price)
- add liquidity directly in the PoolManager on the open (non-Exclusive) builds; remove/donate on every build
- send ETH to native builds only via msg.value of deposit (receive() accepts only the PoolManager)

Discount-listed swap senders (whitelisted executors/routers) UNTRUSTED

- any swap whose PoolManager.swap caller (`sender`) has a live entry gets min(fee, max(ceil(fee*(1 - min(bps,50%))), floor, minFee)) in the listed directions until expiry
- a listed router discounts every end user that routes through it, since the hook sees only the router as sender

Voters admitted by the ballot gate (syncRanges callers) UNTRUSTED

- call FablesLedgerV2.syncRanges for staked ranges of pools the gate says they back; never pausable
- each sync collects v4 fees, splits them staked/unstaked and prices the unstaked backlog's claim fee at that moment

2. Scope

REPOSITORY	FablesFi/fables
COMMIT	b430a02
BRANCH	unknown
SCAN TYPE	Diff Scan
DIFF BASE	main
DIFF TARGET	b430a02a3aa40c45cf1a07dd064901cf78d1eb5f
CHANGED	25 files · 951 lines

Scanned **7** of **7** changed units.

UNIT	FILE

`FablesStableFee` 2 FUNCTIONS
`_calculateDecayingFee`
`_getFee`
	`src/base/FablesStableFee.sol`

`FablesFeeEngine`
WHOLE CONTRACT + 1 FUNCTION
`setCFActive`
	`src/base/FablesFeeEngine.sol`

`FablesLedgerV2`	`src/base/FablesLedgerV2.sol`

`LedgerLibV2`
WHOLE CONTRACT + 1 FUNCTION
`_split`
	`src/libraries/LedgerLibV2.sol`

CHANGED FILES

Regions are contiguous runs of added or modified lines.

FILE	REGIONS	LINES
`src/FablesStableHookLite.sol`	1	1
`src/base/FablesFeeEngine.sol`	4	14
`src/base/FablesLedgerV2.sol`	9	16
`src/base/FablesNativeETHV2.sol`	1	2
`src/base/FablesStableFee.sol`	13	41
`src/base/LedgerTypesV2.sol`	2	2
`src/libraries/LedgerLibV2.sol`	3	5
`test/differential/StableLiteDiffWords.t.sol`	11	91
`test/differential/StableLiteVsStablePair.t.sol`	6	17
`test/fullcycle/FablesHookFullCycle.t.sol`	6	6

FILE	REGIONS	LINES
test/fullcycle/FullCycleBase.sol	1	1
test/gas/EngineGas.t.sol	2	2
test/gas/FullGas.t.sol	1	1
test/invariant/CategoryHandler.sol	1	5
test/invariant/StableFeeEdgesFuzz.t.sol	5	67
test/regression/CFSwitchOnBounds.t.sol	1	339
test/regression/CfAnchorRounding.t.sol	2	2
test/regression/ConfigureCFWhileOn.t.sol	5	5
test/regression/StableWindowFloor.t.sol	1	284
test/unit/engine/EngineEdges.t.sol	2	2
test/unit/engine/FablesFeeEngine.t.sol	14	14
test/unit/full/FullSwaps.t.sol	7	8
test/unit/ledger/LedgerModuleIsolation.t.sol	2	2
test/unit/stable/StableLiteParkedPrice.t.sol	5	7
test/utils/StableRefModel.sol	3	17

3. Ingested Documentation

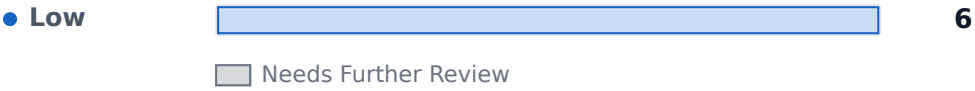
Documentation BugPoCer ingested and used during analysis.

3 DOCUMENTS

DOCUMENT	SOURCE	SIZE
`README.md`	Repository Docs	12,485 chars
`security/pre-audits/README.md`	Repository Docs	1,189 chars
`security/pre-audits/2026-10-29-olympix-bugpocer-0f59859.pdf`	Repository PDFs	35 pages

4. Summary

Scan Started: 02 November 2026 15:43 UTC
Scan Completed: 02 November 2026 17:00 UTC
Report Generated: 02 November 2026 17:08 UTC



YOUR VERDICTS

6 of 6 findings have no verdict yet. Mark each one **True positive**, **False positive** or **Design decision** in the Olympix CLI or dashboard. Your verdicts make future BugPoCer scans of this repository more accurate.

6 distinct issues across 6 findings.

ISSUE	SEVERITY	SITES	ASSESSMENT
Anchor update samples tick manipulated across transactions	Low	1	Needs Further Review
Per-block cached price lets swapper bypass decaying swap-back fee	Low	1	Needs Further Review
Sandwich of swap-back trades via cheap fee on price push	Low	1	Needs Further Review
Unbounded seeder init price can hurt first LPs	Low	1	Needs Further Review
Claim Fee Cut Frontrun By Forced Sync	Low	1	Needs Further Review
Sink Change Misroutes Unsynced Revenue	Low	1	Needs Further Review

SEVERITY

- **High**: Vulnerabilities that enable direct theft or loss of user funds, protocol insolvency, or system-wide failures with realistic attack paths requiring minimal constraints.
- **Medium**: Vulnerabilities that either cause significant impact (fund loss, core functionality breaks) under constrained conditions, moderate impact (temporary DoS, localized losses) under realistic conditions, or minor impact with highly accessible exploit paths.
- **Low**: Vulnerabilities with minimal financial risk that affect non-core functionality, state handling, or minor logic under most conditions, or medium-impact issues requiring admin actions or extreme constraints to exploit.

BADGES ON EACH FINDING

- **Verified**: BugPoCer wrote a proof-of-concept that runs and demonstrates real harm.
- **Needs Further Review**: BugPoCer could not prove it either way; worth a manual review.

5. Findings

5.1. Low

5.1.1. Anchor update samples tick manipulated across transactions

LOW NEEDS FURTHER REVIEW

> **Entry Point:** Cross-Contract Interaction
>
> **Location:** FablesFeeFull.fallback (CF anchor update)

DESCRIPTION

Summary: Anchor update samples tick manipulated across transactions

Mechanism: an attacker could hold a manipulated tick across transactions until the anchor's update interval has passed, so that the update samples the distorted price.

Impact: An attacker holding a manipulated tick across transactions until the anchor update interval passes could skew the anchor's sampled price. Holding a distorted price across blocks exposes the attacker to arbitrage, which costs them capital. If the skewed anchor feeds fee or price logic, LPs or traders could lose value. The item gives no concrete loss, but the hypothetical attack path with stated conditions meets the Medium bar.

Note: Assessed as low impact; reported without a proof of concept.

5.1.2. Per-block cached price lets swapper bypass decaying swap-back fee

LOW NEEDS FURTHER REVIEW

> **Unit Name:** `_calculateDecayingFee`
>
> **Location:** src/base/FablesStableFee.sol: FablesStableFee._calculateDecayingFee

DESCRIPTION

Summary: Per-block cached price lets swapper bypass decaying swap-back fee

Mechanism: an attacker could exploit the per-block cached price: a tiny first swap in a block sets the cache, and later swaps in that same block would then all use the cached price and fee, so moving the price away and back within one block might avoid the decaying swap-back fee

Impact: The description is hypothetical and gives no concrete numbers. If a per-block cached price and fee let a swapper move the price away and back within one block, the swapper avoids the decaying swap-back fee. The fee is the protocol's (and LPs') revenue, and it exists to deter price manipulation. A repeatable bypass would cost the protocol its fees and would remove the anti-arbitrage protection. The item conditions the loss on 'could' and states no amount, but under the rubric a hypothetical attack path with stated assumptions qualifies as Medium.

EVIDENCE

- src/base/FablesStableFee.sol:444-468 (FablesStableFee._calculateDecayingFee)
- src/base/FablesStableFee.sol:120-239

5.1.3. Sandwich of swap-back trades via cheap fee on price push

LOW NEEDS FURTHER REVIEW

> **Unit Name:** `_getFee`
>
> **Location:** src/base/FablesStableFee.sol: FablesStableFee._getFee

DESCRIPTION

Summary: Sandwich of swap-back trades via cheap fee on price push

Mechanism: an attacker can front-run a user's swap-back by pushing the price toward it at the decaying fee, then profit on the backrun, while swaps away are free and can be sandwiched more cheaply

Impact: An attacker front-runs a user's swap-back. They push the price toward the user's trade at the decaying fee, then backrun for profit. The user suffers slippage and loses value. Swaps away from the peg are free and can be sandwiched cheaply. This is a repeatable sandwich that costs the victim more than the fee design should allow. It needs a public mempool, and the item states no chain. The loss is bounded by trade size and slippage, and it exceeds 0.01% for sizeable swaps.

Note: Assessed as low impact; reported without a proof of concept.

EVIDENCE

- src/base/FablesStableFee.sol:392-442 (FablesStableFee._getFee)
- src/base/FablesStableFee.sol:120-239
- src/base/FablesStableFee.sol:549-619

5.1.4. Unbounded seeder init price can hurt first LPs

LOW NEEDS FURTHER REVIEW

> **Entry Point:** Cross-Contract Interaction
>
> **Location:** FablesHook.configureStablePool

DESCRIPTION

Summary: Unbounded seeder init price can hurt first LPs

Mechanism: a SEEDER-chosen initialization price could hurt first LPs, since the hook doesn't bound sqrtPriceX96 and the deposit may lack min-amount protection.

Impact: A semi-trusted SEEDER sets an unbounded initial sqrtPriceX96 for a stable pool. The first LPs deposit without min-amount protection and could be arbitraged at the distorted price, losing principal. The SEEDER is semi-trusted, and the loss depends on the SEEDER acting maliciously or erring. The harm is a direct loss of LP funds with a specific condition, so it meets Medium.

5.1.5. Claim Fee Cut Frontrun By Forced Sync

LOW NEEDS FURTHER REVIEW

> **Unit Name:** `FablesLedgerV2`
>
> **Location:** src/base/FablesLedgerV2.sol: claimFeeFor(), deposit(), _syncRange(); src/libraries/LedgerLibV2.sol: creditFees()

DESCRIPTION

Summary: The docs promise that a claim-fee cut applies to a range's whole unsynced backlog. Anyone can force a sync of any range just before a scheduled or pending cut lands, so the backlog is charged at the old, higher rate and unstaked LPs lose the promised reduction.

Mechanism: `creditFees` prices the backlog with `claimFeeFor(history, claimFeeSyncIndex[id], ...)` and then moves the index to the head. A dust `deposit` (1 liquidity unit), or a holder's `transfer`, runs `_syncRange` on any range. Done before a public `setClaimFee` / `setClaimFeeBps` cut (visible in the AccessManager schedule or mempool), it charges the backlog at the pre-cut rate. The cut then applies only to fees earned afterwards.

Impact: Unstaked LPs pay up to the old rate (at most 20%) instead of the cut rate on the fees that built up before the cut. The difference goes to the claim-fee recipient. The attacker's cost is gas plus a dust deposit per range.

Fix: Apply a cut against existing backlogs too, for example by recording each cut with an effective-from timestamp and letting syncs in a short window before a scheduled cut price at the new rate. Otherwise, drop the "whole backlog" promise.

Note: Assessed as low impact; reported without a proof of concept.

EVIDENCE

- `src/libraries/LedgerLibV2.sol:creditFees()` — computes `feeBps` from the range's old index, then sets `claimFeeSyncIndex[p.id] = history.length`, which fixes the rate for the whole backlog.
- `src/base/FablesLedgerV2.sol:deposit()` — any address can call it with 1 liquidity unit, and it runs `_syncRange(id)` on any configured range.
- `src/base/FablesLedgerV2.sol:claimFeeFor()` — the natspec promises "A cut applies to the whole backlog" but admits "Anyone can force a sync, so a backlog can be priced just before a public cut."
- Obligation: "The claim fee is charged at no more than the lowest rate in force since a range's last sync": the attacker sets when that last sync happens.

5.1.6. Sink Change Misroutes Unsynced Revenue

LOW NEEDS FURTHER REVIEW

> **Unit Name:** `FablesLedgerV2`
>
> **Location:** src/base/FablesLedgerV2.sol: _settleSinks(), setDistributor(), setClaimFee(), setFFFeeSink()

DESCRIPTION

Summary: When a sink changes, only the booked pots of the `settleFirst` currencies go to the outgoing sink. Staked-share fees and claim fees earned under the old sink but not yet synced go to the new distributor or recipient.

Mechanism: `_settleSinks` sweeps only `distributorOwed` / `claimFeeOwed` for the listed currencies. Fees still inside v4 positions are split at each range's next `_syncRange` and booked to pots that are swept to whichever `distributor`, `ffFeeSink` or `claimFeeRecipient` is set at sweep time. Booked pots of currencies left out of `settleFirst` are also redirected.

Impact: On a distributor or claim-fee-recipient change, the outgoing sink's beneficiaries (for example, the previous distributor's voters) lose staked-share and claim-fee revenue earned during their tenure but not yet synced. That revenue goes to the new sink.

Fix: Before a sink change, sync the affected ranges (pass range ids to the setter and run `_syncRange` before `_settleSinks`). Or snapshot each sink's accrual per epoch, so unsynced fees are attributed to the sink in force when they were earned.

Note: This finding may be consistent with the project's documented design; it is flagged for manual review rather than confirmed.

EVIDENCE

- `src/base/FablesLedgerV2.sol:_settleSinks()` — sweeps only the listed currencies' booked pots; the comment admits unsynced fees "are paid to the sinks set when they are swept".
- `src/base/FablesLedgerV2.sol:setDistributor()` — replaces `distributor` / `distributorFF` right after the partial settle; later `sweepDistributor` calls pay the new address.
- `src/libraries/LedgerLibV2.sol:sweepDistributor()` — picks the recipient from the current `distributor` / `ffFeeSink` at sweep time, not when the fees were earned.
- Obligation: "Revenue lost, double-paid, or paid to the wrong sink across a sink change" is the stated impact this behaviour produces.